

MIRA — Next Steps, Outcomes, and Startup Roadmap

Aryan Chavan

<https://github.com/aryaanchavan1-commits/MIRA>

Planning document. Implementation steps name their own decision gates; outcomes are gated or measured, never promised. Numbers trace to committed run artifacts in the repository.

MIRA — Next Steps, Outcomes, and Startup Roadmap

Execution status (updated after the first roadmap pass): S1 DONE — H2 REJECTED on MuSiQue: the K-sweep (n=300, 3 seeds, `data_bench/ncm_ksweep_results.json`) found K=1 at 0.6342 (p=0.0001 vs full), K=2 at 0.6811 (p=0.021), K=3 at 0.6806 (p=0.0002); no K beats full MIRA at 0.6985, so the constellation term is removed from the claim set (it stays behind `ncm.enabled` for the H6 ambiguity line only). S3 DONE — pramāṇa conflict policy implemented (`resolve_conflict`, read-time head annotation, 6/6 tests). S6 DONE — `MemoryEngine.audit()` + `POST /api/audit` (6/6 SDK tests). Packaging: `pyproject.toml` (`mira-memory` 0.2.0). S2 next; S4/S5 BLOCKED (S4 needs the RAM upgrade or a rented box; S5 needs the LongMemEval dataset fetched first — local-only rule forbids auto-download).

Status context for every item below: the framework is implemented and measured (`docs/Framework.md`). Measured base: MuSiQue MRR 0.6985 (n=300, 3 seeds) vs flat-vector 0.4347, BM25 0.3066; HotpotQA 0.9175 vs 0.6774; replay recovers aged memories (+0.0307 MRR); 27/27 test suites; memory self-check 11/11. NCM's default configuration and the Born kernel are currently failing their gates — that is a *feature of the process*, not a failure of the project. Nothing below promises outcomes that are not measured or gated; each step names its decision gate.

1. What to implement next (in priority order)

S1 — H2 K-sweep: decide the constellation term's fate

Build: run `scripts/eval_ncm_pilot.py` with `ncm.top_k ∈ {1, 2, 3}` (K=1 is single-assignment clustering — the degenerate case H2 predicts must differ), n=300, 3 seeds, ~2 h total on the current laptop. **Outcome if K>1 beats K=1:** the overlap kernel is real and gets its paper table. **Outcome if it does not:** H2 is rejected and the term is removed — a published *removal*, which almost no project in the 2026 memory space ever does. **Gate:** paired bootstrap p<0.05 with CI excluding zero.

S2 — H6 ambiguity benchmark: the interference question

Build: a small benchmark of deliberately ambiguous, multi-context queries (same entity in several topic regions; ~100 questions, synthetic from existing corpora). **Outcome:** the first measured answer to “does interference-style combination help ambiguous memory?” — either the Born backend earns a unique niche

or it is cut with evidence. Either way it is a publishable negative-or-positive result. **Gate:** Born must beat linear on ambiguity while not losing elsewhere.

S3 — Pramāṇa-aware contradiction scoring (the differentiator)

Build: when the updater detects a contradiction between two version records, weigh it by pramāṇa class: śabda-vs-śabda (two testimonies) → newer wins, keep both; śabda- vs-pratyakṣa (testimony vs direct input) → direct wins but the testimony is kept as dissent; anumāna never overrides its inputs. Unit tests + one benchmark condition. **Outcome:** conflict resolution by *epistemic source*, not recency alone — no shipped memory system (Mem0, Zep, Letta) does provenance-typed conflict resolution. This is the feature most likely to matter to audit-sensitive users. **Gate:** contradiction-resolution accuracy on a labeled conflict set.

S4 — Scale run: 10⁵–10⁶ nodes

Build: synthetically grow the corpus ×10 and ×100 (copy with perturbed text), measure retrieval p95 latency, memory, and the bounded-fit behavior (`fit_sample` caps are already in). Rent one cloud CPU box if the laptop swaps. **Outcome:** the resource-envelope claim “p95 < 2 s at 10⁵ nodes” becomes measured; million-node claims become honest or stay silent. **Gate:** p95 within envelope, or envelope statement revised downward.

S5 — Port to LongMemEval / LoCoMo

Build: an adapter that loads LongMemEval-S sessions into the store as documents and answers its questions through the SDK (retrieval + extractive answer mode). **Outcome:** numbers comparable against the whole industry (Mem0, Zep, Supermemory all report on it) — credibility that MuSiQue/HotpotQA alone cannot buy. **Gate:** beat the published flat-vector baseline on the same split.

S6 — Audit export (compliance surface)

Build: one endpoint/function: “explain this answer” → JSON with every component score, evidence path, version history, pramāṇa labels, and the current-chain hash. **Outcome:** turns attribution into a *compliance artifact* — the startup wedge (see §3). Cheap: the data already exists in every retrieval.

2. What outcomes no one else is producing

Outcome	Who else does it	Why it is defensible
Per-retrieval component attribution as a product surface	Nobody ships it	Every 2026 memory product is an unattributed vector store behind LLM extraction
A published mechanism-removal ledger (negative results)	Nobody	Vendor benchmarks only rise; MIRA cuts what fails (homeostasis already cut)
Forgetting scored jointly with ranking under a frozen LLM	Nobody as a protocol	Continual-learning work trains weights; agent-memory work ignores retention
Epistemic provenance labels (pramāṇa) on versioned facts	Nobody	Versioning exists (Letta, Zep) but is recency-based, not source-typed

Outcome	Who else does it	Why it is defensible
An interference-kernel (Born-rule) memory result, positive or negative	Nobody measured	QiML exists for classification; memory-retrieval kernels are unmeasured
Local-first, auditable memory for local LLMs	Rarely	The 2026 mainstream is cloud/managed; local-first audibility is an open niche

3. Startup path (three phases, each gated by evidence)

Phase A — Research credibility (now, cost: \$0). Finish S1–S2; publish `steps.pdf`-style honest roadmaps and the removal ledger; keep the repo public with reproducible artifacts. The differentiator *is* the honesty — the market is saturated with benchmark claims it no longer trusts (the LoCoMo-100% disputes). Deliverable: a README whose every number links to a run artifact.

Phase B — Local-first product (after S5). Target: developers running local LLMs (Ollama/LM Studio population) who want memory that explains itself. Ship: `pip install mira-memory` (the SDK), a one-command server, memory adapters for LangChain/LlamaIndex. Run on the user's machine — no cloud bill, which *is* the pitch. Metrics to publish: your benchmark table + p95 latency. Monetization starts as GitHub Sponsors/paid support only.

Phase C — Audit tier (after S3 + S6). The wedge for money: teams that must *justify* what their AI told a customer or an auditor (regulated industries, enterprise copilots). Offer: MIRA as the memory layer with exportable per-answer attribution, version trails, and pramāṇa provenance — an audit trail no vector database provides. Open-core: engine free, hosted multi-tenant audit console + retention policies paid. Do this only if Phase B shows real users; a managed tier is a support liability until then.

What not to build: a fine-tuning story (breaks the frozen-LLM attribution premise), a cloud-only SaaS (abandons the local-first differentiator), an agent framework (you are the memory layer inside others' frameworks).

4. Resources needed

Hardware you already have is enough for S1–S3, S6. The laptop (RTX 3050 4 GB, 16 GB RAM) ran the full 3-seed pilot; the OOM lesson says buy **+16 GB RAM (≈ \$40–60)** before S4 — the single highest-value upgrade.

Need	Spec	Cost	For
RAM upgrade	+16 GB DDR4/5	\$40–60	S4 scaling runs locally
Cloud eval box (rental, not owned)	8 vCPU / 32 GB CPU-only	~\$0.10–0.40/h	million-node runs, CI benchmarks
Demo VPS (Phase B)	2 vCPU / 4 GB	\$10–25/mo	public demo server; embeddings run on CPU
No GPU cloud needed	—	\$0	retrieval is CPU-bound by design; local LLMs are the user's
Software	Python, SQLite, FAISS, FastAPI, pandoc/Chrome	\$0	all already in the repo, permissive licenses

Need	Spec	Cost	For
People	solo founder is viable through Phase B	time	Phase C needs +1 (support/integrations)

Total cash to reach Phase B: **under \$100**. That is only possible because the product is local-first — say so in the pitch.

5. 90-day plan

Weeks	Do	Gate/exit
1–2	S1 K-sweep + S2 ambiguity benchmark	constellation decision recorded (rescue or removal)
3–5	S3 pramāṇa contradiction scoring + tests	conflict-set accuracy vs recency baseline
5–7	S5 LongMemEval adapter	comparable table row vs published baselines
7–9	S4 scale run (rent box if needed)	measured p95/memory envelope
9–10	S6 audit export + pip packaging + adapter	installable <code>pip</code> package; demo server
11–13	Publish everything; launch post (HN/r/LocalLLaMA); first 10 users	≥10 external installs; feedback ledger

6. Honest risks

- **NCM may die entirely** (S1/S2 both fail): the framework survives — attribution, versioning, dynamics, and the Indic methods remain the product — but the research program’s headline shrinks. Plan the pitch around the framework, not one mechanism.
- **Benchmark credibility is asymmetric**: one honest number can be drowned out by vendor claims; mitigate by porting to LongMemEval where third parties can rerun you.
- **Solo-founder bandwidth**: the 90-day plan assumes ~15 h/week; cut S4 first if not.
- **Local-LLM UX debt**: Ollama/LM Studio integration is unplanned engineering.